Decoding Transformer Architecture

Wednesday, July 29, 2026 7:55 PM

Transformer takes a sequence of tokens as input and produces a sequence of tokens as output. That is it. **At a very high level**, it is a **tokens-in**, **tokens-out** machine.

The **Transformer** was introduced in **2017** in the famous research paper "Attention Is All You Need" by researchers at Google. It changed everything.

The Architecture Has Two Halves:

- The Encoder - this half reads and understands the input
- The Decoder - this half generate the output

Think of it like a conversation between a reader and a writer.
The reader (encoder) reads a document carefully and creates detailed notes.
The writer (decoder) uses those notes to write a new document.

The **reader** focuses on **understanding**
The **writer** focuses on **producing.**

Decoding step 1: Turning words into Numbers

Tokenization: The input text is split into small pieces called tokens.

Embedding: Each token is then converted into a vector of numbers called an embedding.

Decoding step 2: Adding Word Order

Positional Encoding: It adds a vector of numbers to each embedding that represents the position of the word.

Ex: "I love AI" and "AI love I"

Decoding step 3: The Attention Mechanism

This is the heart of the Transformer. The attention mechanism is the single most important idea that makes everything work.

Attention -> Try to learn what is your relationship with all the others.

sentence: "The cat sat on the mat because it was tired."

When the model processes the word "it", it needs to figure out what "it" refers to. Is it the cat? Is it the mat? The attention mechanism helps the model focus more on "cat" and less on "mat" because "cat" is more relevant to "it" in this context.

How Attention Works

Each word is converted into three things:

- Query (Q) - what this **word is looking for**

- Key (K) - what this **word can offer**
- Value (V) - the **actual information this word carries**

The Attention Formula:

The core formula for **Scaled Dot-Product Attention** is:

```
Attention(Q, K, V) = softmax(Q x K^T / sqrt(d_k)) x V
```

- `Q` is the Query matrix
- `K` is the Key matrix
- `V` is the Value matrix
- `K^T` is the transpose of the Key matrix
- `d_k` is the dimension of the Key vector
- `sqrt` means square root

Setting Up: From Words to Vectors

Let's take a simple sentence with 3 words:
"I love AI"

we will use very small vectors of size 4 (i.e. `d_emb = 4`).

```
"I"    → [1.0, 0.0, 1.0, 0.0]
"love" → [0.0, 1.0, 0.0, 1.0]
"AI"   → [1.0, 1.0, 0.0, 0.0]
```

Input matrix **X** of the Shape **3x4** (**3 words**, each with **4 numbers**)

```
X = | 1.0  0.0  1.0  0.0 |   ← "I"
    | 0.0  1.0  0.0  1.0 |   ← "love"
    | 1.0  1.0  0.0  0.0 |   ← "AI"
```

Creating Q, K, and V Matrices

Create three separate matrices: Q (Query). K (key), and V (Value).

How do we create them?

We multiply the input matrix `X` with three separate
weight matrices: `W_Q`, `W_K`, and `W_V`. These weight matrices are learned during training.

```
Q = X x W_Q
K = X x W_K
V = X x W_V
```

`d_k = 3` (i.e. we want Q, K, V vectors of size 3). So each weight matrix has the shape `4 x 3` (input dimension 4, output dimension 3).

```
W_Q = | 1  0  1 |
      | 0  1  0 |
      | 1  0  0 |
      | 0  1  1 |

W_K = | 0  1  0 |
      | 1  0  1 |
      | 0  0  1 |
      | 1  1  0 |

W_V = | 1  0  0 |
      | 0  1  0 |
      | 0  0  1 |
      | 1  1  0 |
```

Computing Q = X x W_Q:

```
Q = | 1.0  0.0  1.0  0.0 |     | 1  0  1 |
    | 0.0  1.0  0.0  1.0 |  x  | 0  1  0 |
    | 1.0  1.0  0.0  0.0 |     | 1  0  0 |
                               | 0  1  1 |
```

For the first row ("I"): `[1*1 + 0*0 + 1*1 + 0*0, 1*0 + 0*1 + 1*0 + 0*1, 1*1 + 0*0 + 1*0 + 0*1]` = `[2, 0, 1]`

For the second row ("love"): `[0*1 + 1*0 + 0*1 + 1*0, 0*0 + 1*1 + 0*0 + 1*1, 0*1 + 1*0 + 0*0 + 1*1]` = `[0, 2, 1]`

For the third row ("AI"): `[1*1 + 1*0 + 0*1 + 0*0, 1*0 + 1*1 + 0*0 + 0*1, 1*1 + 1*0 + 0*0 + 0*1]` = `[1, 1, 1]`

```
Q = | 2  0  1 |   ← "I"
    | 0  2  1 |   ← "love"
    | 1  1  1 |   ← "AI"
```

Computing K = X x W_K:

For the first row ("I"): [1*0 + 0*1 + 1*0 + 0*1, 1*1 + 0*0 + 1*0 + 0*1, 1*0 + 0*1 + 1*1 + 0*0] = [0, 1, 1]
For the second row ("love"): [0*0 + 1*1 + 0*0 + 1*1, 0*1 + 1*0 + 0*0 + 1*1, 0*0 + 1*1 + 0*1 + 1*0] = [2, 1, 1]
For the third row ("AI"): [1*0 + 1*1 + 0*0 + 0*1, 1*1 + 1*0 + 0*0 + 0*1, 1*0 + 1*1 + 0*1 + 0*0] = [1, 1, 1]

```
K = | 0  1  1 |   ← "I"
    | 2  1  1 |   ← "love"
    | 1  1  1 |   ← "AI"
```

Computing V = X x W_V:

For the first row ("I"): [1*1 + 0*0 + 1*0 + 0*1, 1*0 + 0*1 + 1*0 + 0*1, 1*0 + 0*0 + 1*1 + 0*0] = [1, 0, 1]
For the second row ("love"): [0*1 + 1*0 + 0*0 + 1*1, 0*0 + 1*1 + 0*0 + 1*1, 0*0 + 1*0 + 0*1 + 1*0] = [1, 2, 0]
For the third row ("AI"): [1*1 + 1*0 + 0*0 + 0*1, 1*0 + 1*1 + 0*0 + 0*1, 1*0 + 1*0 + 0*1 + 0*0] = [1, 1, 0]

```
V = | 1  0  1 |   ← "I"
    | 1  2  0 |   ← "love"
    | 1  1  0 |   ← "AI"
```

Note: In real models, these weight matrices are not set by hand. They are learned during training. We are using simple numbers here for the sake of understanding.

Computing Attention Scores (Q x K^T):

First, we need K^T (transpose of K).

```
K^T = | 0  2  1 |
      | 1  1  1 |
      | 1  1  1 |
```

Now, let's compute Q x K^T:

```
Q x K^T = | 2  0  1 |     | 0  2  1 |
          | 0  2  1 |  x  | 1  1  1 |
          | 1  1  1 |     | 1  1  1 |
```

For "I" attending to all words:

- "I" → "I": 2*0 + 0*1 + 1*1 = 1
- "I" → "love": 2*2 + 0*1 + 1*1 = 5
- "I" → "AI": 2*1 + 0*1 + 1*1 = 3

For "love" attending to all words:

- "love" → "I": 0*0 + 2*1 + 1*1 = 3
- "love" → "love": 0*2 + 2*1 + 1*1 = 3
- "love" → "AI": 0*1 + 2*1 + 1*1 = 3

For "AI" attending to all words:

- "AI" → "I": 1*0 + 1*1 + 1*1 = 2
- "AI" → "love": 1*2 + 1*1 + 1*1 = 4
- "AI" → "AI": 1*1 + 1*1 + 1*1 = 3

```
            "I"   "love"   "AI"
Scores = |  1      5       3  |   ← "I"
         |  3      3       3  |   ← "love"
         |  2      4       3  |   ← "AI"
```

Scaling the Scores:

Now, we divide every score by sqrt(d_k).

In our example, d_k = 3 (the dimension of our Key vectors), so sqrt(3) = 1.732.

Why do we scale?

- If the dot product values are too large.
- The softmax function in the next step will produce very extreme values (close 0 to 1).
- This makes it difficult for the model to learn. Scaling keeps the values in a manageable range.

```
Scaled Scores = Scores / sqrt(3)
```

```
                    "I"      "love"     "AI"
Scaled Scores = |  0.577    2.887     1.732  |   ← "I"
                |  1.732    1.732     1.732  |   ← "love"
                |  1.155    2.309     1.732  |   ← "AI"
```

Applying Softmax:

The softmax formula for a value x_i in a row is:

```
softmax(x_i) = e^(x_i) / sum(e^(x_j) for all j in that row)
```

Here, e is the mathematical constant (**approximately 2.718**). Do not worry about the formula too much. We will compute it step by step.

For the "I" row: [0.577, 2.887, 1.732]

- e^0.577 = 1.781
- e^2.887 = 17.940
- e^1.732 = 5.651
- Sum = 1.781 + 17.940 + 5.651 = 25.372

Attention Weights:

- "I" → "I": 1.781 / 25.372 = 0.070
- "I" → "love": 17.940 / 25.372 = 0.707
- "I" → "AI": 5.651 / 25.372 = 0.223

For the "love" row: [1.732, 1.732, 1.732]
All values are equal, so each word gets equal attention:

- "love" → "I": 0.333
- "love" → "love": 0.333
- "love" → "AI": 0.333

For the "AI" row: [1.155, 2.309, 1.732]

- e^1.155 = 3.174
- e^2.309 = 10.063
- e^1.732 = 5.651
- Sum = 3.174 + 10.063 + 5.651 = 18.888

Attention Weights:

- "AI" → "I": 3.174 / 18.888 = 0.168
- "AI" → "love": 10.063 / 18.888 = 0.533
- "AI" → "AI": 5.651 / 18.888 = 0.299

The final **Attention Weight Matrix** is:

```
                      "I"     "love"    "AI"
Attention Weights = | 0.070    0.707    0.223 |   ← "I"
                    | 0.333    0.333    0.333 |   ← "love"
                    | 0.168    0.533    0.299 |   ← "AI"
```

Computing the Final Output (Attention Weight x V):

Now, we multiply the Attention Weight Matrix with the Value matrix V to get the final output.

```
Output = Attention Weights x V
```

```
Output = | 0.070  0.707  0.223 |     | 1  0  1 |
         | 0.333  0.333  0.333 |  x  | 1  2  0 |
         | 0.168  0.533  0.299 |     | 1  1  0 |
```

For "I":

- 0.070*1 + 0.707*1 + 0.223*1 = 1.000
- 0.070*0 + 0.707*2 + 0.223*1 = 1.637
- 0.070*1 + 0.707*0 + 0.223*0 = 0.070

For "love":

- 0.333*1 + 0.333*1 + 0.333*1 = 0.999
- 0.333*0 + 0.333*2 + 0.333*1 = 0.999
- 0.333*1 + 0.333*0 + 0.333*0 = 0.333

For "AI":

- 0.168*1 + 0.533*1 + 0.299*1 = 1.000
- 0.168*0 + 0.533*2 + 0.299*1 = 1.365
- 0.168*1 + 0.533*0 + 0.299*0 = 0.168

This was all about computing the final output. Now, let's put it all together.

To master **self-attention, multi-head attention,** and the full Transformer math in depth,

Putting It All Together

Let's summarize the entire computation in one place:
Step 1: Start with the input embeddings (X).
Step 2: Multiply X with weight matrices to get Q, K, and V.
Step 3: Compute attention scores by multiplying Q with the transpose of K (Q x K^T).
Step 4: Scale the scores by dividing by sqrt(d_k).
Step 5: Apply softmax to get attention weights (probabilities).
Step 6: Multiply attention weights with V to get the final output.

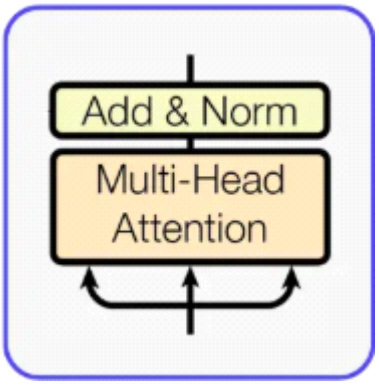
The entire process in one formula:

```
Attention(Q, K, V) = softmax(Q x K^T / sqrt(d_k)) x V
```

Now, every part of this formula is clear.

This is how the math behind Query, Key, and Value works in the Attention mechanism.

Multi-Head Attention in Transformers



What is Multi-Head Attention?

Multi-Head Attention is a mechanism that runs many self-Attention operations in parallel, each with its own set of Q, K, and V projections, and then combines their outputs into single richer representation.

Multi-Head Attention = Multi + Head + Attention

Head means one independent Self-Attention.
Multi means run many such heads in parallel.
Attention means each head decides how much to focus on every other token.

The Self Attention formula is:
Attention(Q, K, V) = softmax((Q . K^T) / sqrt(d_k)) . V

Here, we have done the following:

- Multiply Q and K^T to find how much each token relates to every other token.
- Divide by sqrt(d_k) to keep the numbers stable.
- Apply softmax to turn the scores into probabilities.
- Multiply by V to get a weighted sum of values.

Why do we need Multi-Head Attention?

I Love AI
This is a short sentence, but there are many different relationships hidden in it.

- I is the subject of the sentence.
- love is the action.
- AI is the object that the subject loves.
- I and love form a subject-verb relationship.
- love and AI form a verb-object relationship.
- I and AI are connected through the meaning of love.

Now, the question is: can a single attention head learn all of these relationships at the same time?

The answer is no.

A single Self Attention head can only learn one type of relationship at a time. If one head is busy learning the subject-verb link, it cannot also learn the verb-object link with the same focus.

So, here comes Multi-Head Attention into the picture.

With multiple heads, each head can focus on a different type of relationship. One head can learn the **subject-verb link.** Another head can learn the **verb-object link.** Another head can learn the **long-range link** between the subject and the object. And so on.

This way, the model gets a much richer understanding of the same sentence.

Step-by-Step working of Multi-Head Attention:

Assume, We have input sequence with d_model = 512 and we want to use h = 8 heads.

Here, d_model is the size of the vector that represents each token

every token in the sentence is converted into a list of numbers.

Step 1: We start with the input embeddings. Every token in the sentence is converted into a vector of size d_model.

Step 2: We split the work across heads. We divide d_model by the number of heads. So, each head works with a smaller dimension.

d_k = d_model / h = 512 / 8 = 64

Each head gets a dimension of 64 for its Q, K, and V.

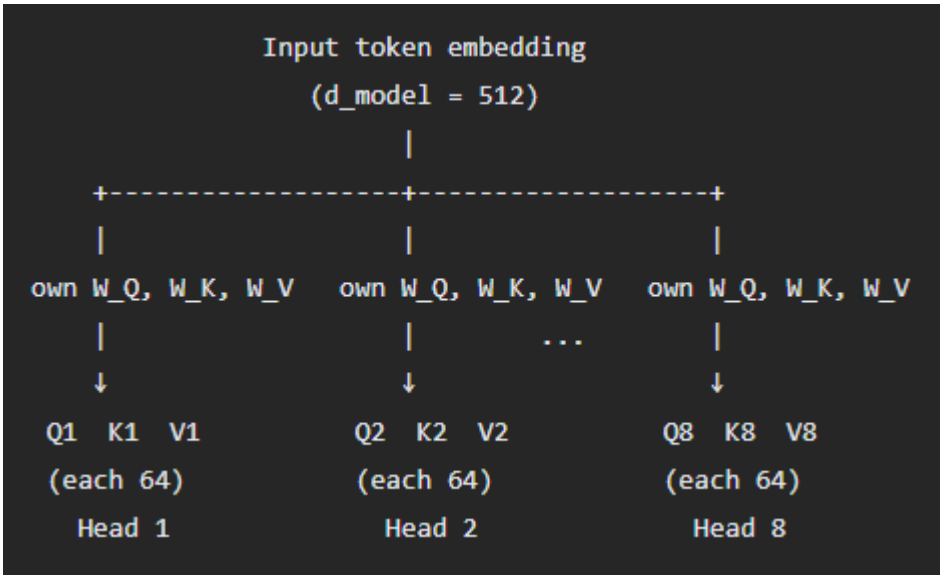
Step 3: For every head, we use its own learned linear projections to create the Q, K, and V vectors.

- **Q_i = Input x W_i_Q**
- **K_i = Input x W_i_K**
- **V_i = Input x W_i_V**

Here, W_i_Q, W_i_K, and W_i_V are the weight matrices for head i that the model learns during training.

Note: Every head has its own set of weight matrices. This is what allows different heads to learn different patterns.

We can visualize how each head creates its own Q, K, and V as below:



Step 4: Each head now runs Self Attention independently using its own Q_i, K_i, and V_i.

head_i = Attention(Q_i, K_i, V_i) = softmax((Q_i . K_i^T) / sqrt(d_k)) . V_i

All 8 heads do this in parallel.

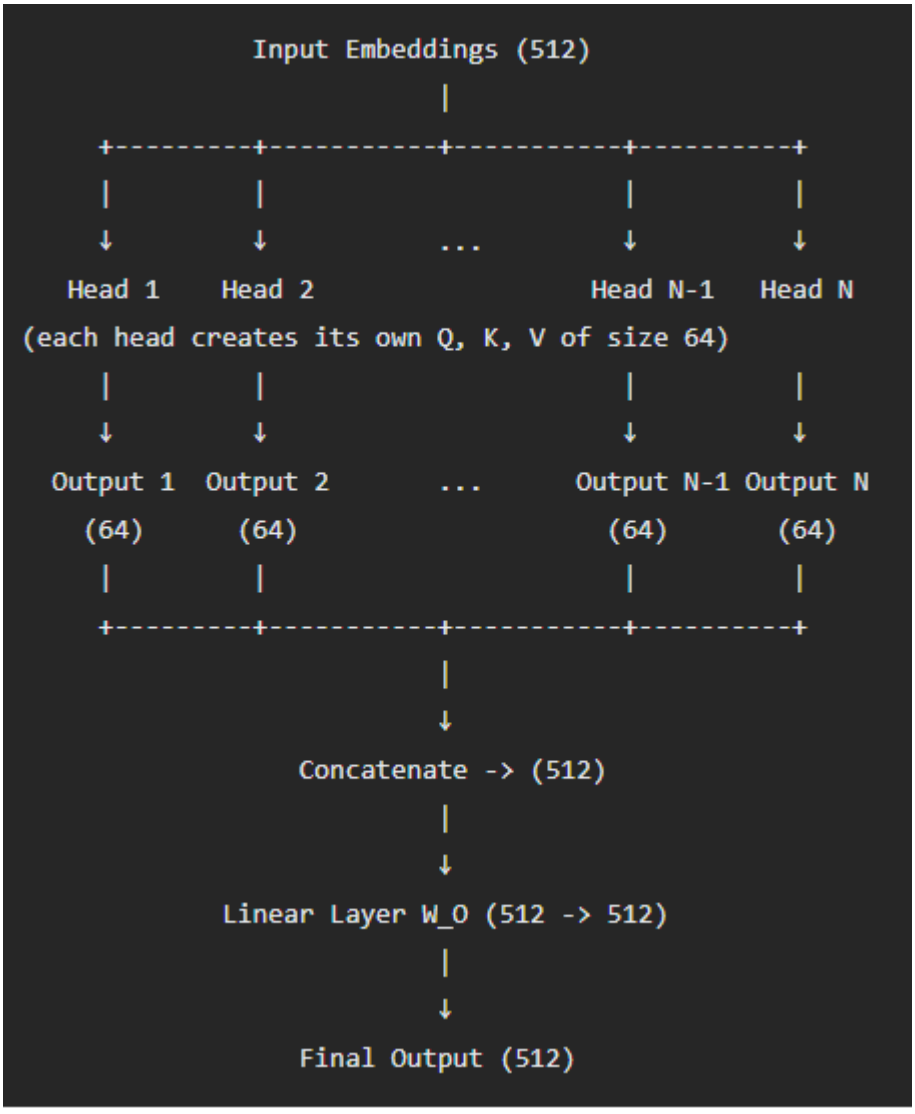
Step 5: We concatenate the output of all heads side by side.
Concat(head_1, head_2, ..., head_8)

After concatenation, we get a vector of size h x d_k = 8 x 64 = 512, which is the same as d_model. So, the output dimension matches the input dimension.

Step 6: We pass the concatenated output through one more learned linear layer to mix the information from all heads.

MultiHead(Q, K, V) = Concat(head_1, ..., head_8) . W_O

Here, W_O is a learned weight matrix. This is the final output of Multi-Head Attention.



Multi-Head Attention = many Self Attentions running in parallel + one final mixing step.

Where Multi-Head Attention is used?

Multi-Head Attention is used in three places inside a Transformer.

Encoder Multi-Head Self-Attention: Every input token attends to every other input token.

Decoder Masked Multi-Head Self-Attention: Every Output token attends only to itself and the tokens before it. The future tokens are hidden using a mask, because the decoder generates one taken at time and must not look at future tokens.

Decoder Multi-Head Cross Attention: The decoder attends to the encoder output. Here 'Q' comes from the decoder, and 'K' and 'V' come from the encoder. This is the bridge between the encoder and the decoder.

Advantages of Multi-Head Attention:

Richer representations - Every head learns a different pattern, so the final output captures many

relationship at once.

Parallel computation - All head run in parallel, So it is very fast on modern Gpus.

Same total cost - head uses a smaller dimension (d_model/h), the total computation is similar to one big attention with the full dimension.

Better learning - model can capture both short-range and long-range patterns at the same time.

Flexibility - Different heads can specialize in different things, syntax, meaning, position, long-range links, and etc.

The original Transformer used 8 heads. Modern Large Language Models go much higher. Many LLMs today use 32, 64, or even more attention heads in every layer, because more heads can capture more relationships in the data.

Having more heads also makes inference more memory-hungry, so many modern LLMs use a variant called **Grouped Query Attention** that keeps the quality close to Multi-Head Attention while using far less memory.